

## Module-3 Combinational Circuits.

PAGE NO.:

DATE: / /

### ★ Compact Truth table representation.

We know that:

| a | b | $f(a,b)$ |
|---|---|----------|
| 0 | 0 | x        |
| 0 | 1 | y        |
| 1 | 0 | z        |
| 1 | 1 | w        |

and

$$f = x\bar{a}\bar{b} + y\bar{a}b + z a\bar{b} + w a b$$

Now, from this, we can easily create CTT given expression. ✓

Ex:  $f = \bar{a}b + a\bar{b} + \bar{c}ab$

then C-TT is

| a | b | $f(a,b,c)$ |
|---|---|------------|
| 0 | 0 | 0          |
| 0 | 1 | 1          |
| 1 | 0 | 1          |
| 1 | 1 | 0          |

This is C-TT, std. TT will have 8 rows as  $f(a,b,c)$ .

→ CTT is just another representation of the function.

We can easily convert std. T.T. to Compact truth table as:

Ex:  $f(a,b,c)$

| a | b | c | $f(a,b,c)$ |
|---|---|---|------------|
| 0 | 0 | 0 | 1          |
| 0 | 0 | 1 | 0          |
| 0 | 1 | 0 | 1          |
| 0 | 1 | 1 | 1          |
| 1 | 0 | 0 | 1          |
| 1 | 0 | 1 | 0          |
| 1 | 1 | 0 | 0          |
| 1 | 1 | 1 | 0          |

← std TT. →

$f(a,b,c)$

| a | b | $f(a,b,c)$ |
|---|---|------------|
| 0 | 0 | $\bar{c}$  |
| 0 | 1 | 1          |
| 1 | 0 | $\bar{c}$  |
| 1 | 1 | 0          |

OR

$f(a,b,c)$

| a | c | $f(a,b,c)$ |
|---|---|------------|
| 0 | 0 | 1          |
| 0 | 1 | b          |
| 1 | 0 | $\bar{b}$  |
| 1 | 1 | 0          |

CTT

CTT<sub>2</sub>

PAGE NO. : \_\_\_\_\_

DATE : / /

Given Exp  $\rightarrow$  CTT ✓  
 $\hookrightarrow$  STT ✓

Given CTT  $\rightarrow$  Exp. ✓  
 $\hookrightarrow$  STT. ✓

Given STT  $\rightarrow$  Exp ✓  
 $\hookrightarrow$  STT ✓

You know all

Table can be made more compact.

for Ex: for prev. ex only

| a | f                 |
|---|-------------------|
| 0 | $b + \bar{c}$     |
| 1 | $\bar{b} \bar{c}$ |

OR

| b | f         |
|---|-----------|
| 0 | $\bar{c}$ |
| 1 | $\bar{a}$ |

or

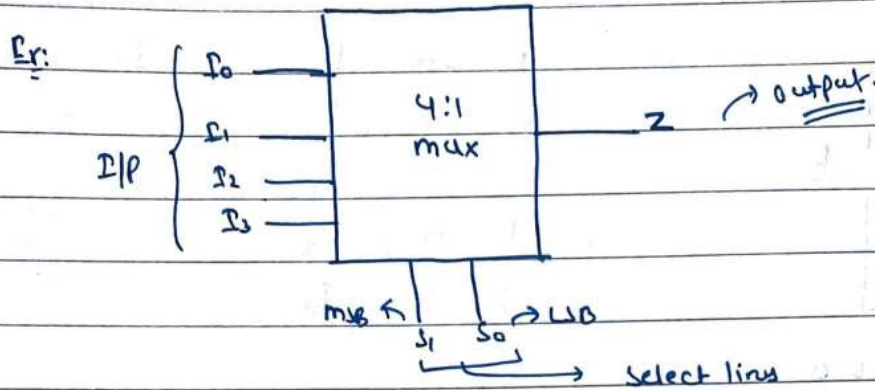
| c | f                   |
|---|---------------------|
| 0 | $\bar{a} + \bar{b}$ |
| 1 | $\bar{a} b$         |

Note that exp. written by any above table will be same.

$\rightarrow$  CTT is used to represent logic circuits like mux, encoder,  
etc



★ Multiplexer:  $n \times 1$  circuit with  $\log_2 n$  select lines  $\equiv$  bridge

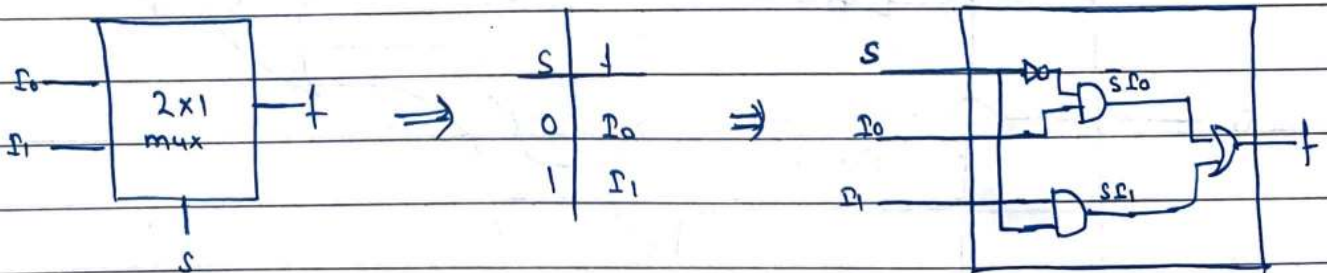


$Z$  will be  $I_2$  if  $S_0$  is 0 &  $S_1$  is 1 ✓

Mux ( $\equiv$  Multiplexer) is a data selector i.e. selects one i/p on the basis of select lines.

→  $2^n \times 1$  mux has  $n$  select lines ✓

→ It is the fixed repr. of mux to be  $2^n \times 1$  not  $2^n \times 2$  bcz, it is def. like this ✓



Block Diagram

Truth table

Actual Circuit.

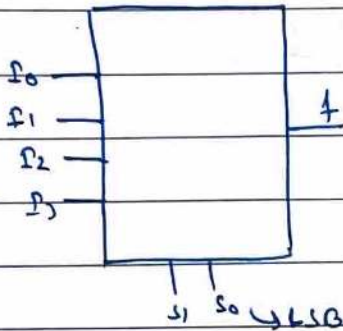
Ex:  $\bar{S}I_0 + SI_1$   
Std. TT. will have 4 rows ✓

PAGE NO. : \_\_\_\_\_

DATE: / /

Similarly, for 4x1 mux:

Block Diagram:



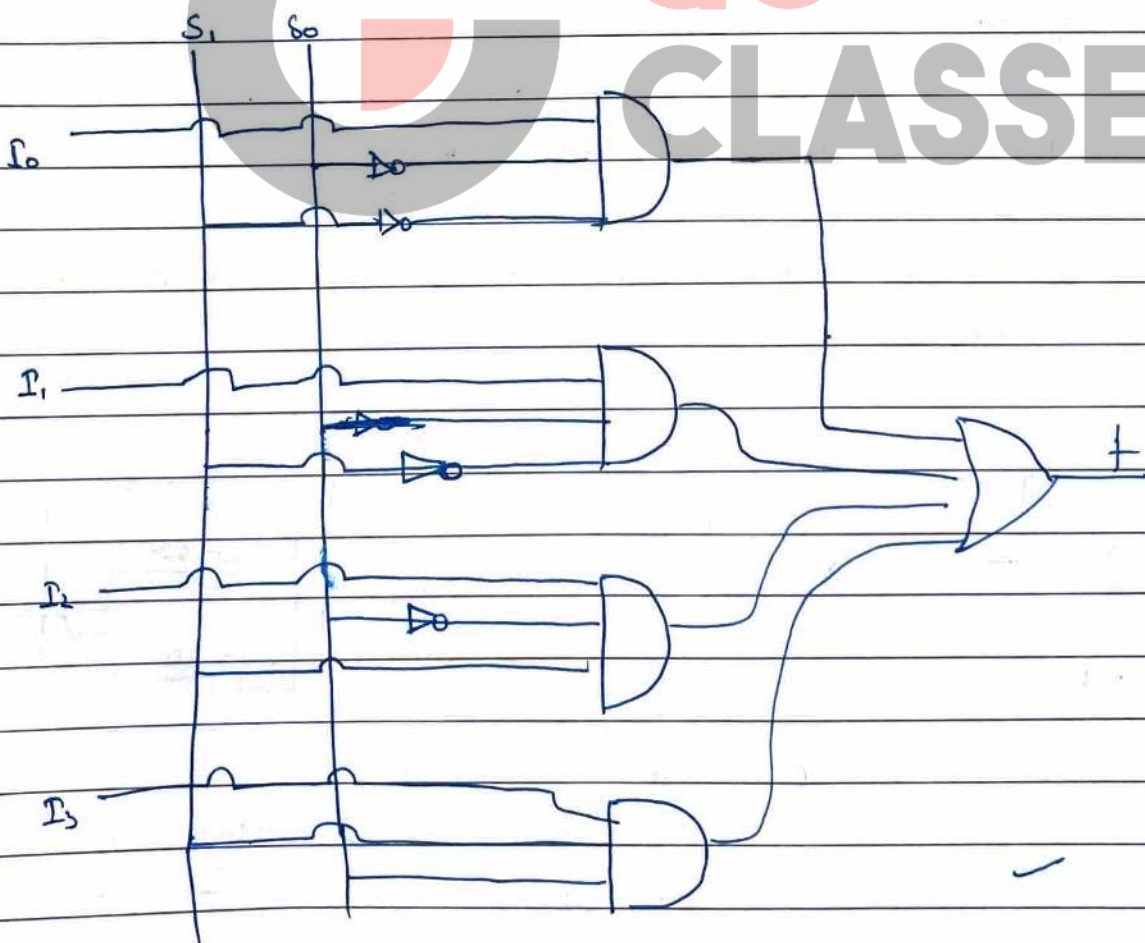
C.T.T

| $S_1$ | $S_0$ | $f$   |
|-------|-------|-------|
| 0     | 0     | $I_0$ |
| 0     | 1     | $I_1$ |
| 1     | 0     | $I_2$ |
| 1     | 1     | $I_3$ |

Exp.

$$f = \bar{S}_0 \bar{S}_1 I_0 + \bar{S}_0 S_1 I_1 + S_0 \bar{S}_1 I_2 + S_0 S_1 I_3$$

Actual Circuit:



If  $S_0 = 1$ ,  $S_1 = 0$  then  $f$  will be at  $I_2$ .



PAGE NO.:

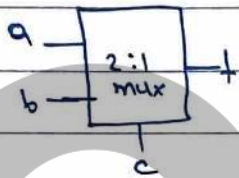
DATE: / /

Generally,

$$f = \sum_{k=0}^{2^n-1} m_k I_k$$

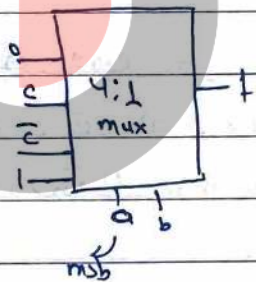
where  $m_k$  is  $k^{\text{th}}$  minterm.✓  $I_k$  is  $k^{\text{th}}$  input.So  $k^{\text{th}}$  i/p will be selected as o/p iff  $k^{\text{th}}$  minterm is True.⇒ Realisation / Implementation of functions using mux:

Ex: 1



$$f = \bar{c}a + cb$$

Ex: 2



$$f(a,b,c) = \bar{a}\bar{b}0 + \bar{a}b\bar{c} + a\bar{b}\bar{c} + ab1$$

$$= b\bar{c} + a\bar{c}$$

→ (on simplifying)

This is how we can easily get which function is implemented on seeing the mux.

mux → fun ✓

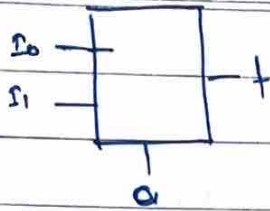
Now we will see fun → mux:

⇒ OR using mux:

Attempt 0



No sense as using OR gate.



We want  $f = a + b$ .

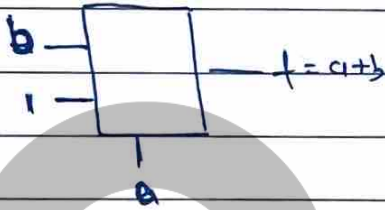
$$f \text{ is } \bar{a}I_0 + aI_1$$

If we take  $I_1$  as 1 &  $I_0$  as  $b$  then,

$$f = \bar{a}b + a$$

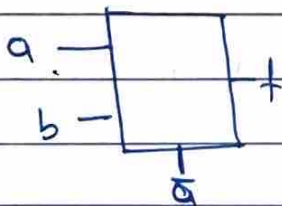
$$= \underline{a + b}$$

So,



How to guess this? best way is get SOP of function you want to implement i.e.  $a + b$  here =  $ab + \bar{a}b + a\bar{b}$ , now for  $\bar{a}$  we have  $b$  and for  $a$  we have  $(b + \bar{b})$  i.e. 1.  $\checkmark$

Many other ways to implement:

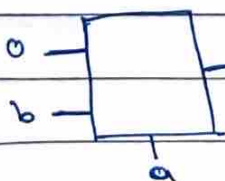


$$f = \bar{a}a + \bar{a}b = \bar{a} + \bar{a}b \Rightarrow \underline{a + b}$$

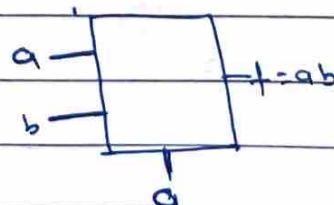
but it's not ideal as an extra inverter is req. to input  $\bar{a}$ .

$\Rightarrow$  ~~AND~~ AND gate using mux:

$$f = ab \rightarrow \text{already in SOP so 0 for } \bar{a} \text{ \& } b \text{ for } a$$



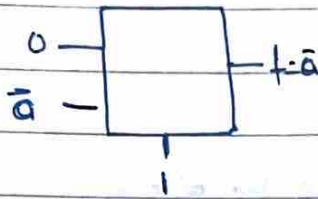
OR



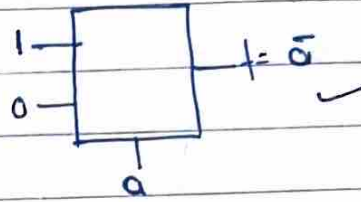
$$\Rightarrow \bar{a}\bar{a} + ab = \underline{ab} \checkmark$$



⇒ Inverter using mux:



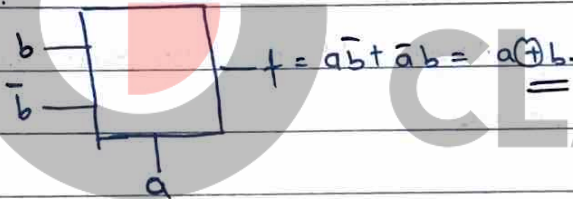
Not a good ckt. as already req. inv.



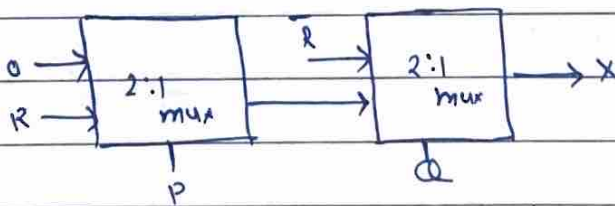
⇒ XOR using mux:

$$f = a\bar{b} + \bar{a}b$$

So.



Ques: What's the expression of X?



$$X = \bar{a}\bar{R} + a(PR)$$

$$= \bar{a}\bar{R} + \underline{\underline{PaR.}} \quad \checkmark$$

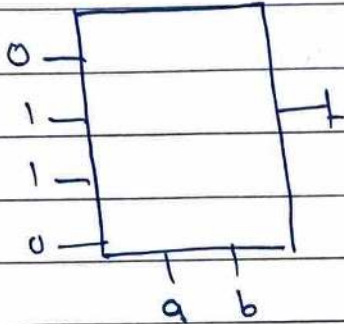
Karan

PAGE NO : \_\_\_\_\_

DATE : / /

Ques: Implement  $a \oplus b$  using 4:1 mux.

$$\Rightarrow f(a,b) = \bar{a}b + a\bar{b}$$

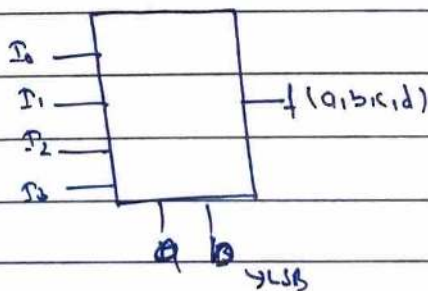


Directly can be given  
by truth table easily. ✓

→ If asked with  $2^n \times 1$  mux, put one select line (ms) as 0 and last  $n$  if as don't cares.

$f = a \oplus b \oplus c$  is one if exact odd ones are there i.e. 1 or 3

So  $f = \bar{a}\bar{b}c + a\bar{b}\bar{c} + \bar{a}b\bar{c} + abc$  ✓

Ans: Implement  $f(a,b,c,d) = ad + bc + \bar{a} + b + c$  by following mux:

$$f = ad + bc + \bar{a}\bar{b}$$

(Method-1)

when  $a=0, b=0, I_0 = \bar{c}$

when  $a=1, b=0, I_1 = d$

when  $a=0, b=1, I_2 = c$

when  $a=1, b=1, I_3 = d + c$  ✓

OR can do with CSOP also (Method-2) ✓

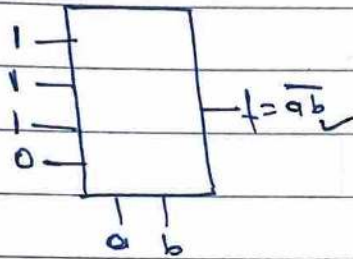


PAGE NO.:

DATE: / /

**NOTE**  $\longleftrightarrow$  Single  $2^n:1$  mux is enough (w/o any other gate) for any  $n$ -variable function as seen in xor ex. on prev. page. We just write T.T. as i/p.

**Ex:** for  $f(a,b) = \overline{a}b$ , only  $4:1$  mux is needed



put var on select line & truth value on input lines.

$\rightarrow$  Single  $2^n:1$  mux is not always enough for  $n$ -var function.

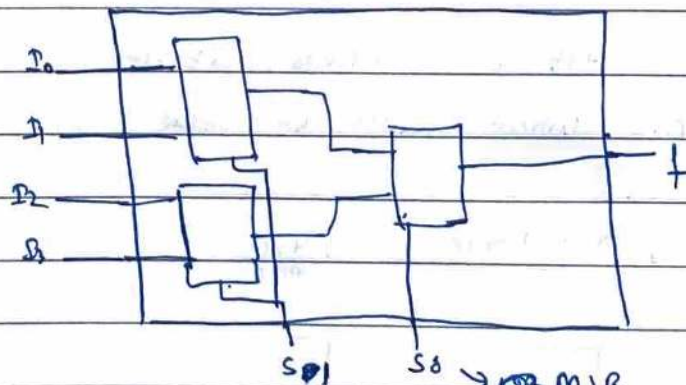
**Ex:** Can't implement  $X \oplus Y$  by  $2:1$  mux w/o inverter. ✓

However some  $n$ -var. fun. can be implemented but not all.

**NOTE**  $\longleftrightarrow$  Single  $2^n:1$  mux + Single inverter are enough for any  $n$ -variable function. (Think of Compact truth table).

★ Building larger mux from smaller mux:

$\rightarrow$   $4 \times 1$  mux using  $2 \times 1$  mux only



!!!

LSB will network

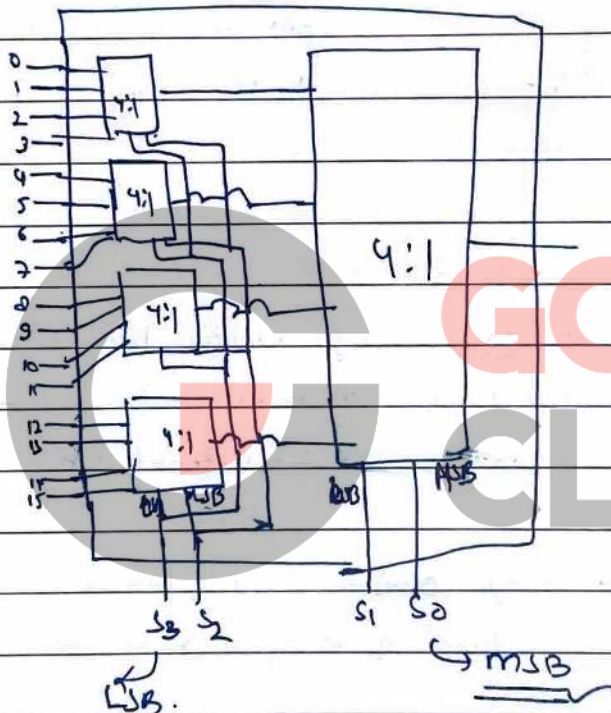
PAGE NO : \_\_\_\_\_

DATE : / /

Check all settings like so msb is req. So LSB will not work. ✓

→ 8x1 mux can be derived from  $\frac{n}{2} (n+1)$  2x1 muxs.

→ 16x1 mux from 4x1 muxs



Order will be

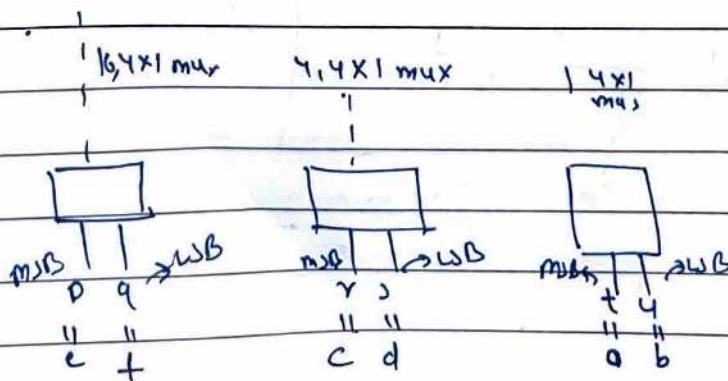
$S_3, S_2, S_1, S_0$

msb ← → lsb

msb

How to Choose select lines? Just try what you get in diff settings & what were you expecting.

Ques: In 6x1 mux with 6 select lines a b c d e f with a as msb and f as lsb. what will be value of f q r s t u below?



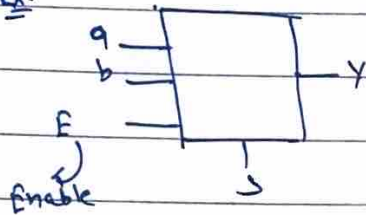
(work on this)



### ⇒ Multiplexer with Enable Input:

Enable if is present in every comb ckt. It's like on/off switch,  
Circuit works if enable is 1, else ckt will not work.

Ex:



PT-2

| E | S | Y |
|---|---|---|
| 0 | X | 0 |
| 1 | 0 | a |
| 1 | 1 | b |

→ mux is idle, if  $E=0$

→ mux working normally

↳ Compact PT.

Std. will have 2<sup>+</sup> rows.

as  $Y$  is  $f(a, b, E, S)$

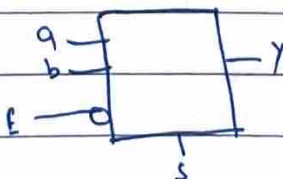
Now, in every exp. we calculate we can incl. with  $E$  so that it work only if  $E$  is 1.

$$\text{Here } F = f(\bar{S}a + Sb) = E\bar{S}a + ESb$$

There are two types of enable:

→ Active low enable: Circuit work if  $E=0$ , and if  $E=1$  circuit is idle.

→ Active high enable: Circuit work if  $E=1$ , (Circuit is active when enable is high), and if  $E=0$  circuit is idle. Explained this above. Default



mux with active low enable.

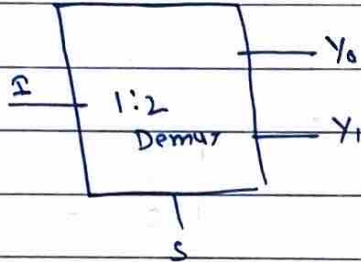
→ Easily can add one more line in circuit to implement enable. When don't want ckt to work always, it only needs when enable is 1.

★ Demultiplexer (Demux): Another combinational circuit.

Opposite of mux. Not much imp.

1 i/p and 2<sup>n</sup> o/p lines.

Ex:

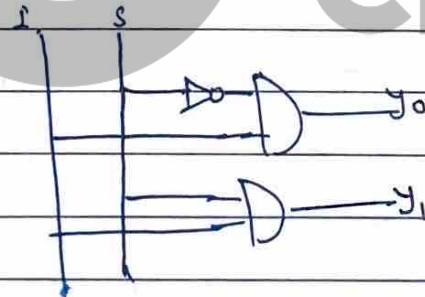


T.T:

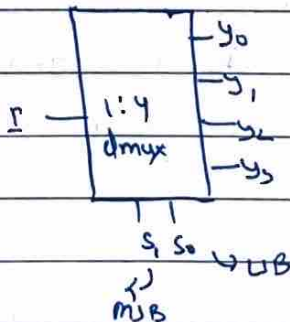
| S | Y <sub>0</sub> | Y <sub>1</sub> |
|---|----------------|----------------|
| 0 | I              | 0              |
| 1 | 0              | I              |

Exp:  $Y_0 = \bar{S}I$ , and,  $Y_1 = SI$

Imp:



Similarly:



| S <sub>1</sub> | S <sub>0</sub> | Y <sub>0</sub> | Y <sub>1</sub> | Y <sub>2</sub> | Y <sub>3</sub> |
|----------------|----------------|----------------|----------------|----------------|----------------|
| 0              | 0              | I              | 0              | 0              | 0              |
| 0              | 1              | 0              | I              | 0              | 0              |
| 1              | 0              | 0              | 0              | I              | 0              |
| 1              | 1              | 0              | 0              | 0              | I              |

CTT:

$$Y_0 = \bar{S}_1 \bar{S}_0 I, Y_1 = \bar{S}_1 S_0 I, Y_2 = S_1 \bar{S}_0 I, Y_3 = S_1 S_0 I$$



PAGE NO.:

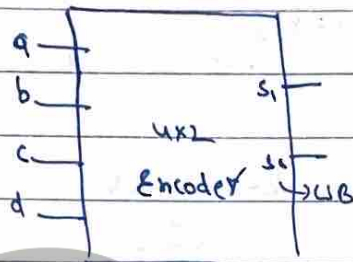
DATE: / /

★ Encoder: Encoding means giving codes. So basically, encoder does encoding, i.e. binary encoding.

Gives code to  $2^n$  peoples using  $n$  bits.

It is of size  $2^n : n$  and no concept of select lines.

Ex:



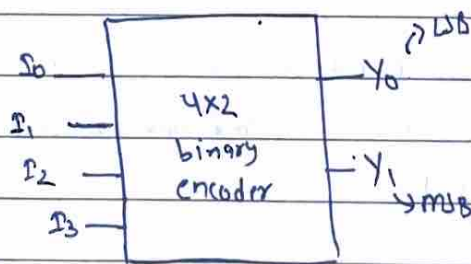
$$2^n \rightarrow I/P, n$$

$$n \rightarrow O/P$$

When d will be 1 then  $s_1 s_2 = 1$   
 c " " 1 "  $s_1 s_2 = 0$

There are two types of encoders:

- 1) Binary Encoders: Here, we allow only one i/p to be 1 at a time i.e.  $a=1$  &  $b=1$  is not possible.



Now, truth table is

| $I_0$                              | $I_1$ | $I_2$ | $I_3$ | $Y_1$ | $Y_0$ |
|------------------------------------|-------|-------|-------|-------|-------|
| 1                                  | 0     | 0     | 0     | 0     | 0     |
| 0                                  | 1     | 0     | 0     | 0     | 1     |
| 0                                  | 0     | 1     | 0     | 1     | 0     |
| 0                                  | 0     | 0     | 1     | 1     | 1     |
| Remaining<br>combs.<br>can't occur | x     | x     | x     | x     | x     |

one hot encoded  $\rightarrow$  exactly  $\pm 1$  bit is

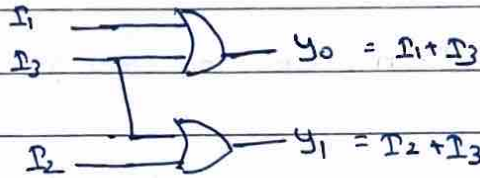
Made By - Karan Agrawal (GATE CS 2024 AIR 102)

PAGE NO. :

DATE : / /

$$Y_1 = I_2 + I_3, \quad Y_0 = I_1 + I_3$$

So, actual imp:



K-map for  $Y_0$  :

| $I_3 \backslash I_1 I_2$ | 00 | 01 | 11 | 10 |
|--------------------------|----|----|----|----|
| 0                        | 0  | 0  | 0  | 0  |
| 1                        | 0  | 1  | 1  | 1  |
| 2                        | 0  | 1  | 1  | 1  |
| 3                        | 0  | 1  | 1  | 1  |

$$Y_0 = I_2 + I_1$$

$$Y_0 = \bar{I}_2 \bar{I}_0$$

K-map for  $Y_1$ :

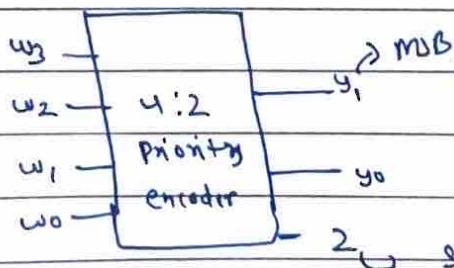
| $I_3 \backslash I_1 I_2$ | 00 | 01 | 11 | 10 |
|--------------------------|----|----|----|----|
| 0                        | 0  | 0  | 0  | 0  |
| 1                        | 0  | 1  | 1  | 1  |
| 2                        | 0  | 1  | 1  | 1  |
| 3                        | 0  | 1  | 1  | 1  |

$$Y_1 = I_2 + I_3$$

$$Y_1 = \bar{I}_0 \bar{I}_1$$

An useful example of encoder is Octal to binary converter (8X3 encoder), it is given in one-hot encoded form (i.e. out of 8 bits 1 bit is 1).

b) Priority Encoder: Here if more priority i/p is 1, then rest do not matter.



let  $w_0$  = lowest priority

&  $w_3$  = highest "

Someone used notation.



Here we have an extra o/p which is <sup>only</sup> 0 when all inputs are 0.

(x → don't care)

| <u>I.T.</u> | $w_3$ | $w_2$ | $w_1$ | $w_0$ | $y_1$ | $y_0$ | <del>0</del> 2 |
|-------------|-------|-------|-------|-------|-------|-------|----------------|
|             | 1     | x     | x     | x     | 1     | 1     | 1              |
|             | 0     | 1     | x     | x     | 1     | 0     | 1              |
|             | 0     | 0     | 1     | x     | 0     | 1     | 1              |
|             | 0     | 0     | 0     | 1     | 0     | 0     | 1              |
|             | 0     | 0     | 0     | 0     | 0x    | 0x    | 0              |

$$y_1 = w_3 + w_2, \quad y_0 = w_3 + w_2 + w_1$$

→ Here 2 is not enable i/p. It is o/p which is 1 when someone need attention at  $y_0, y_1$  can be zero when we need attention or no one need attention. So to dist. this we have 2.

$$2 = w_3 + w_2 + w_1 + w_0. \quad \checkmark$$

↳ 1 if <sup>attent</sup> anyone is 1.

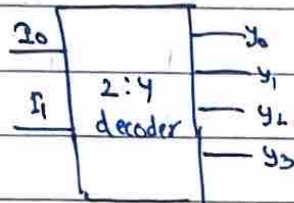
PAGE NO.:

DATE: / /

Imp. mux &amp; decoder

★ Decoders: If we are encoding we also need to decode.It has  $n$  inputs and  $2^n$  outputs. Exactly one of  $2^n$  ops is True.

Ex:



T, T2

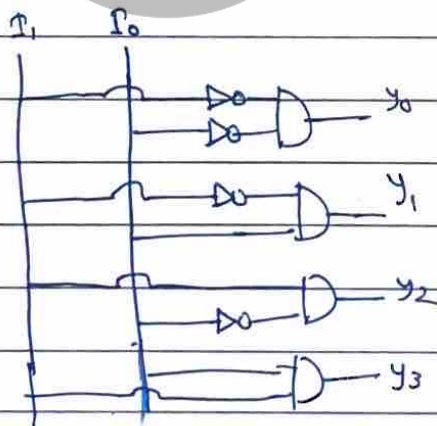
| $I_1$ | $I_0$ | $Y_0$ | $Y_1$ | $Y_2$ | $Y_3$ |
|-------|-------|-------|-------|-------|-------|
| 0     | 0     | 1     | 0     | 0     | 0     |

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 1 | 0 | 1 | 0 | 0 |
|---|---|---|---|---|---|

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 1 | 0 |
|---|---|---|---|---|---|

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 1 | 0 | 0 | 0 | 1 |
|---|---|---|---|---|---|

$$Y_0 = \bar{I}_0 \bar{I}_1, Y_1 = \bar{I}_1 I_0, Y_2 = I_1 \bar{I}_0, Y_3 = I_1 I_0$$



Decoder can also have enable i/p (active low or active high)

→ Decoder is analogue to Receptionist who tells you path to dept, once you tell dept. code. ✓



PAGE NO.:

DATE: / /

→ Practical ex. of a decoder is binary to Octal encoder.  
i/p is 3 digit binary o/p is particular decimal as one hot encoding.

⇒ Active low decoder: Default decoder was active high decoder  
where exact one o/p was one.

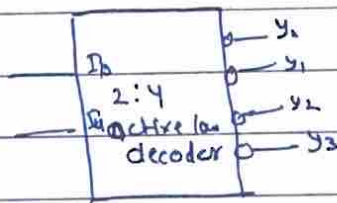
In active low decoder exactly one o/p will be 0 and  
all other are 1.

T.T.

| MSB   |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|
| $I_1$ | $I_0$ | $Y_0$ | $Y_1$ | $Y_2$ | $Y_3$ |
| 0     | 0     | 0     | 1     | 1     | 1     |
| 0     | 1     | 1     | 0     | 1     | 1     |
| 1     | 0     | 1     | 1     | 0     | 1     |
| 1     | 1     | 1     | 1     | 1     | 0     |

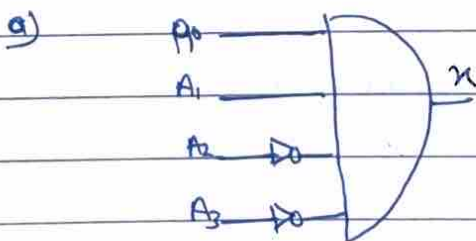
It can be implemented using NAND gates

It is repr. as:



NAND decoder  $\Rightarrow$  Active low dec.

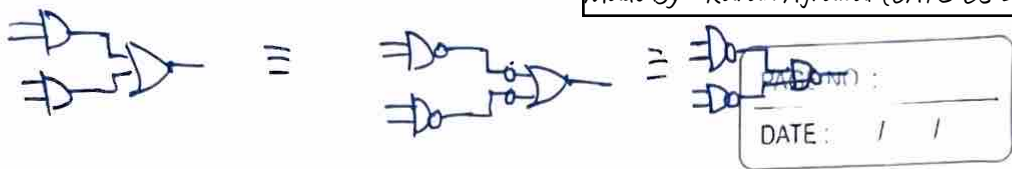
Ques: Which binary no. is detected by following circuits give  
 $A_0$  is LSB &  $A_3$  is MSB.



⇒ ~~Ans.~~ ~~no. is~~ ~~more than~~

0011

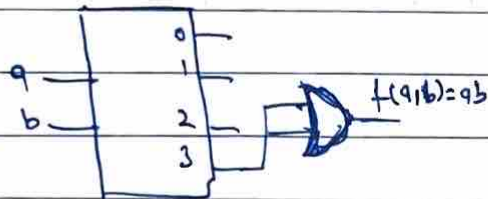
↳ bcz. for this  $n=1$



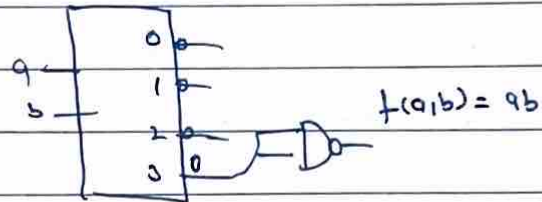
⇒ function realization using decoder:

Ex:  $f(a,b) = ab$  using

a) active high decoder

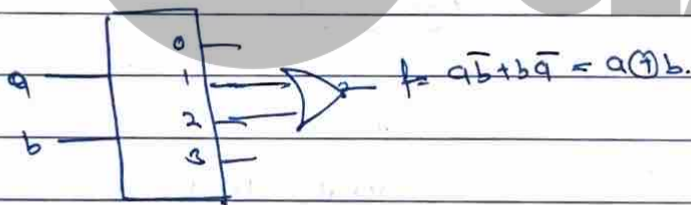


b) active low decoder

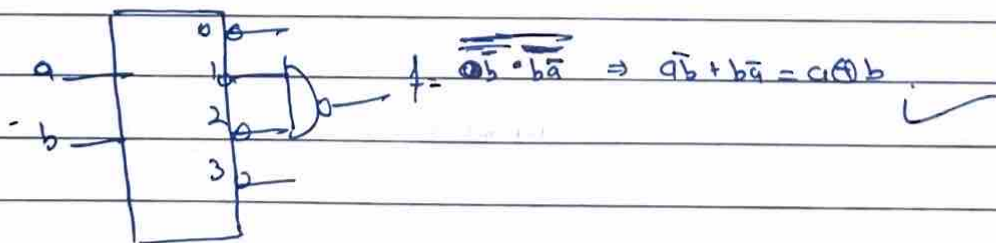


Remember to use always OR with active high decoder i.e. AND decoder and use NAND with active low i.e. NAND decoder.

Ques: Implement  $f(a,b) = a \oplus b$  using active high decoder



b) active low decoder



So, for implementing n var function we need  $2^n$  decoder and 1 NAND or OR Gate.

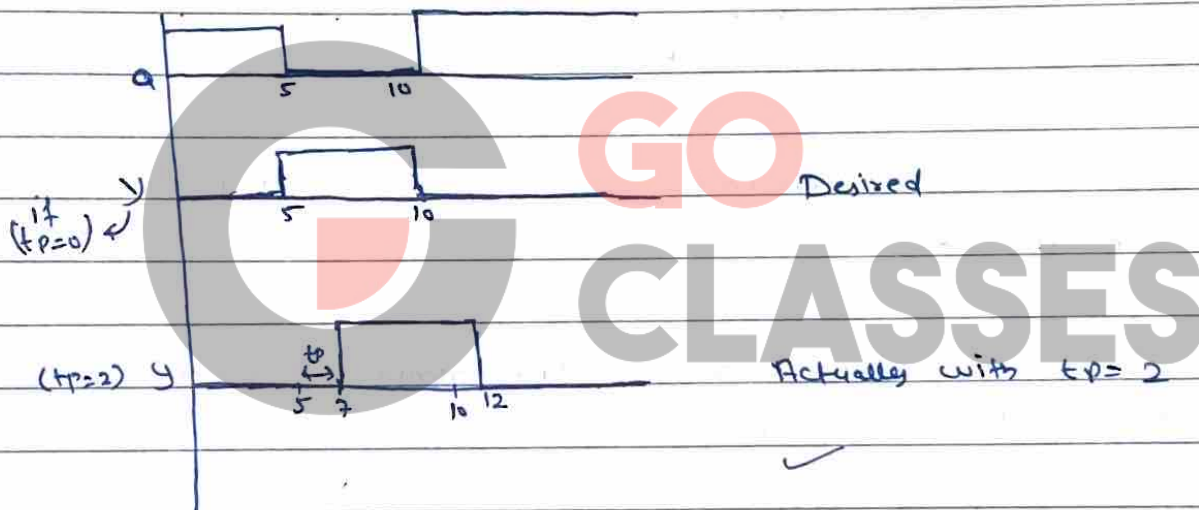


★ Propagation delay: When we change i/p, then not instantly o/p changes, it takes very little delay to react changes. bco in circuit there are physical elements that work. This delay is called propagation delay of gate.

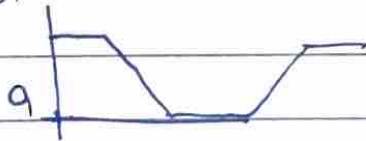
Ex: let  $t_p = 2\text{ ns}$  (prop. delay). for



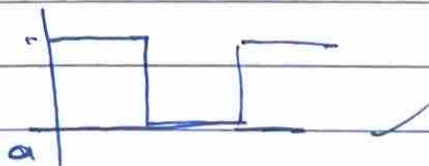
then timing diagram will be:



Practically,



this happens, but as told in m-1, we consider range of voltage as high and range as low. So, theoretically it will be eq. to:



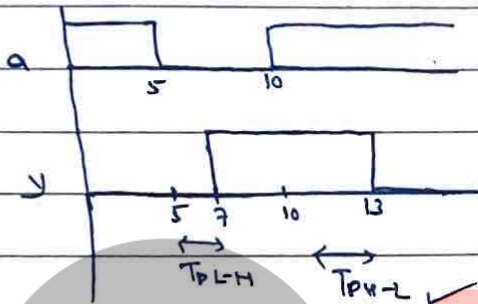
PAGE NO : \_\_\_\_\_

DATE : / /

→ Practically  $T_p$  when going from low to high &  $T_p$  when going from high to low can be different

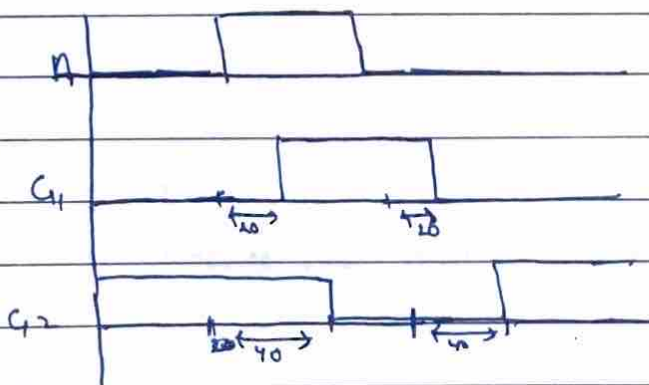
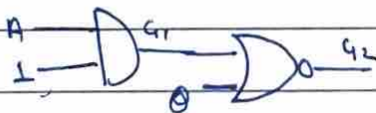
Ex:   $T_{pL-H} = 2\text{ ns}$  &  $T_{pH-L} = 3\text{ ns}$

then



But theoretically we'll assume  $T_{pL-H} \equiv T_{pH-L} \equiv T_p$  until unless given in question.

Ex:  $T_p$  for each gate is  $20\text{ ns}$ , then timing diagram for below circuit is as:





PAGE NO. :

DATE : / /

⇒ Propagation delay of a circuit:

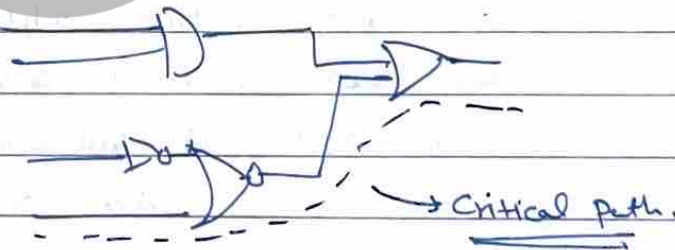
Imp definitions:

- Maximum prop. delay is the longest delay b/w an i/p changing value and the o/p changing value.
- The path that causes this delay is called the critical path. It imposes limit on max. speed of circuit.
- Pd of circuit is Pd of critical path.

So Pd of circuit is max. delay possible (on longest route).

Ex: Prev. ex's Prop. delay is  $20+20 = \underline{40\text{ns}}$ .

Ex:2 Every gate's delay is  $10\text{ns}$ , then delay of prev circuit is:



→ We assume Pd for wire to be 0, but practically it exists.

→ For delay just shift waveform to right by Pd time.



## Adders (last comb. det. of syllabus)

- We have seen some fundamental circuits i.e. mux, demux, encoder and decoder. Now we'll design most fundamental thing required and that is adder.
- We will see addition of unsigned numbers. After that we'll use same circuit for signed numbers also. (small part of lecture).
- Adder forms basic component of ALU. It has four basic elementary operation possible:  $0+0=0$ ,  $1+0=1$ ,  $1+1=10$  &  $0+1=1$
- We may need to add 3 bits if carry is 1 & that will be done using full adders.

★ Half Adder: It deals with 2 bits addition.  
i.e. take two bits as input & gives sum & carry bits as output. In full adders we use prev. carry but here we don't care about that.

Add two bits → Use half adder

Add 2 bits & a carry → Use full adder

Add 2 - multibit numbers → Use a) Ripple Carry adder  
b) Carry lookahead adder



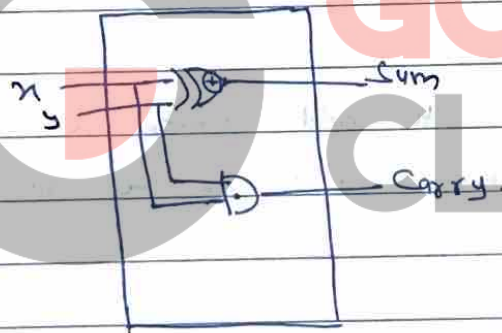
Truth table of half adder:

| I/P |   | O/P   |     |
|-----|---|-------|-----|
| X   | Y | Carry | Sum |
| 0   | 0 | 0     | 0   |
| 0   | 1 | 0     | 1   |
| 1   | 0 | 0     | 1   |
| 1   | 1 | 1     | 0   |

$$S = x\bar{y} + y\bar{x} = x \oplus y$$

$$C = xy$$

So, implementation of half adder is so easy



Half adder

Ques: Add. 011 & 101 using half adder

⇒ Can't do bcz

$$\begin{array}{r} 011 \\ + 101 \\ \hline 1101 \end{array}$$

3bit req

only 2<sup>nd</sup> LSB can be added.

Ques: If 't' is the prop. delay of AND, OR, NOR, EXOR gate etc, then delay of half adder is?

⇒ t (for 2<sup>nd</sup> t & for 1<sup>st</sup> t)

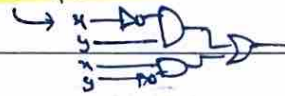
PAGE NO. : \_\_\_\_\_

DATE : / /

→ For last que, we can't take AND-OR implementation of XOR unnecessarily to ↑ the prop. delay.

Que: Delay of AND, OR, NOT gate i.e. basic gates is  $d$ , then delay of half adder is?

⇒  $3d$  i.e.  $d$  for carry and  $2d$  for XOR i.e. sum.



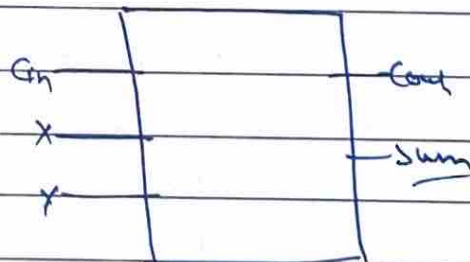
Que: In above que, if i/p available in original & complemented form then delay is?

⇒  $2d$  ( $d$  for sum and  $d$  for carry)

★ Full adder: Circuit to add 3 bits. (2 of, no & 1 of, carry)  
It takes 3 i/p & gives two o/p

Truth table:

| i/p |   |     | o/p |      |
|-----|---|-----|-----|------|
| x   | y | cin | Sum | cout |
| 0   | 0 | 0   | 0   | 0    |
| 0   | 0 | 1   | 1   | 0    |
| 0   | 1 | 0   | 1   | 0    |
| 0   | 1 | 1   | 0   | 1    |
| 1   | 0 | 0   | 1   | 0    |
| 1   | 0 | 1   | 0   | 1    |
| 1   | 1 | 0   | 0   | 1    |
| 1   | 1 | 1   | 1   | 1    |



Block diagram

cout is 1 when atleast two input are 1 &

Sum is 1 when odd i/p are 1



$\oplus \equiv \oplus$  when both one is not possible

PAGE NO.:

DATE: / /

$$S \equiv \bar{x}\bar{y}c_{in} + \bar{x}yc_{in} + x\bar{y}\bar{c}_{in} + xy c_{in} = x \oplus y \oplus c_{in}$$

$$C_{out} = xy + yc_{in} + xc_{in}$$

For Sum we have two equations shown above.

For  $C_{out}$  we have many equations:

$$\# C_{out} = xy + c_{in}x + c_{in}y \quad - (i)$$

$$C_{out} = xy + c_{in}(x+y) \quad - (ii)$$

$$\# C_{out} = xy + c_{in}(x \oplus y) \quad - (iii)$$

$$C_{out} = xy \oplus c_{in}(x \oplus y) \quad - (iv)$$

$$C_{out} \neq xy \oplus c_{in}(x+y) \quad - (v) \quad \text{X}$$

→ Equation (v) is not valid for  $C_{out}$ ,  
because if  $x=1, y=1, c_{in}=1$ , then  $1 \oplus 1 = 0$ , but it must be 1.

→ Note that  $xy$  &  $(x \oplus y)c_{in}$  can't be true at same time bcz  
if  $xy=1$  then  $x \oplus y = 0$ , so that's why  $xy \oplus (x \oplus y)c_{in}$  works.

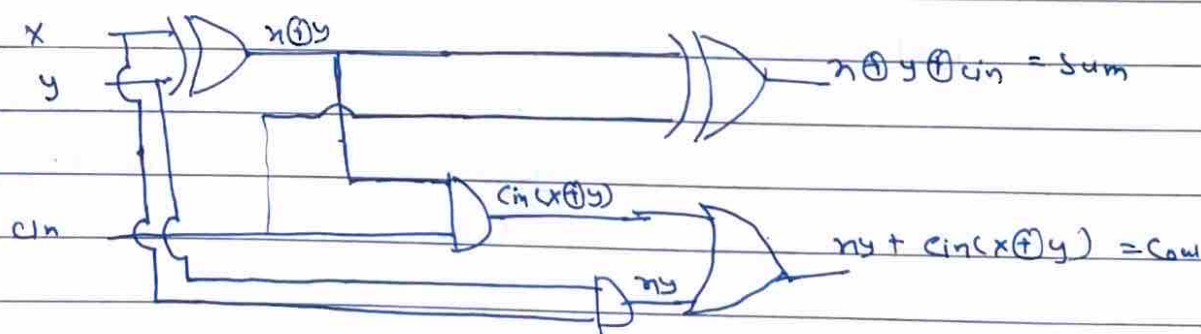
→ Any fun. can be written as exor of true minterms the way  
we wrote OR of true minterms

$$\text{Ex: } f = \sum m(1,3) = \bar{a}b + ab = \bar{a}b \oplus ab$$

This is because two minterms cannot be 1 at same time

So OR & EXOR will behave same.

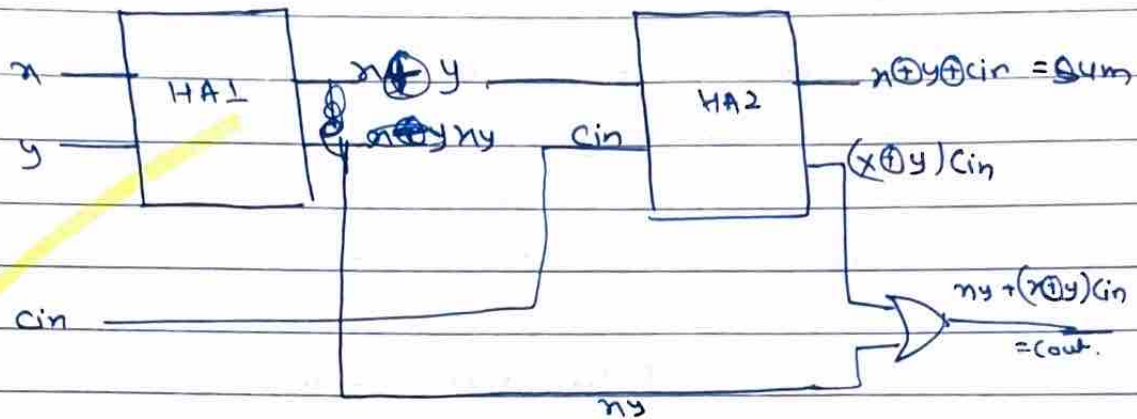
→ AND, OR, EXOR implementation



PAGE NO.:

DATE: / /

→ Two half adder & 1 OR implementation



→ Simple AND, OR, Not implementation can easily be made from expression derived. (2 level for Cout & 3 level for sum)

Ques: Can we use only HA to implement FA?

⇒ Yes, we use 3 HAs (1 in place of OR),  $C_{out} = xy + (x \oplus y)C_{in}$ .

Note:  $\text{Sum} = x \oplus y \oplus C_{in} = x \odot y \odot C_{in}$ .

★ Ripple Carry Adder: Uses many FAs to add multibit binary number i.e. does actual addition.

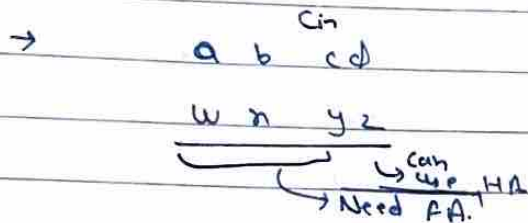
⇒ Delay of FA:

Only basic gates (AND, OR, NOT, NAND, NOR), & each gates' delay is  $t$  thus it is in 3 $t$  (2 $t$  for carry & 3 $t$  for sum)

(Think from Basic Equation of Sum & Carry)

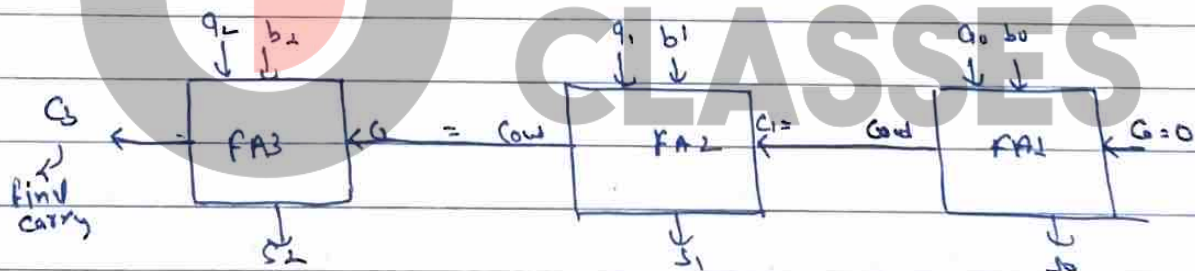


→ We design binary adder by cascading full adder with Cout of prev. f.adder as cin of current FA. Ripple carry & carry lookahead are two designs of n-bit parallel adder.



→ RCA (Ripple Carry adder) resembles our human addition computation.

Ex: For 3 bit numbers say  
 $a_2 a_1 a_0 + b_2 b_1 b_0$



This is how RCA works.

→ In RCA, carry ripples through the FAs to Cout finally. **n-bit binary adder needs just n FAs.**

→ Initial  $s_0, s_1, s_2$  are invalid i.e. unavailable for us then after some delay  $s_0$  becomes available, then  $s_1$  & finally  $s_2$  &  $C_2$ .

→  $s_2, C_2$  depends on  $C_1$ ,  $C_1$  depends on  $C_0$  &  $C_0$  depends on  $C_{in}$ . So, all bits become available only after carry propagates through the whole circuit.

Que: Assume every FA is created using basic Gates; then what's the delay of RCA if every gate has delay of 'd'?

⇒ We will get  $S_0$  after  $3d$  &  $C_0$  at  $2d$ .

Now, we get  $S_1$  after  $3d+2d$  and  $C_1$  at  $2d+2d$ .  
 ↳ to get  $C_0$  used in  $S_1$   
 $3d+2d$

Similarly  $S_2$  at  $7d$  &  $C_2$  at  $2d+2d+2d = 6d$

Finally  $S_3$  at  $6d+2d = 8d$  and  $C_4$  at  $8d$ .  
 ↳ force ↳ freedom

$C_n$  at  $2nd$  and  $S_n$  at  $(2n+1)d$

Que: FA is implemented using 2HA & 1 OR gate. Delay of AND, OR, NOT is  $d$  and that of XOR is  $2d$ . then RCA delay is?

⇒ Time to get sum:  $4d$  (critical path, 2 XOR of 4As)

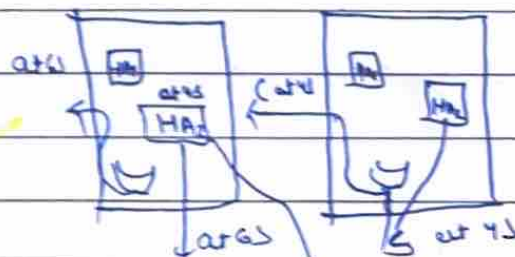
Time to get carry:  $4d$  (critical path → 1 XOR, 1 AND, 1 OR)

Now  $S_0$  at  $4d$ ,  $C_1$  at  $4d$

$S_1$  at  $6d$ ,  $C_2$  at  $6d$

↳ bcz already  $4d$ , comp. will be done, only left is dep. of  $C_1$

~~$S_n = 2nd$~~ ,  $C_n = 2nd$



Ready before getting  $C_{prev}$ .



⇒ Drawbacks of RCA:

\* Total delay is proportional to #bits.

- Performance degrades for large  $n$ .
- Bottleneck is carry which propagates seq. from one stage to next.

→ Drawbacks of RCA can be overcome if we somehow gen. all carry bits in advance before actual addition, then after the initial delay, all bits can be summed up in parallel. Time compl. shift from  $O(n)$  to  $O(1)$ , but h/w comp.  $n$ .

★ Carry Look Ahead (CLA) adder: In this we gen. all carries initially and then sum up all bits in one go.

Initially we have  $a_3, a_2, a_1, a_0, b_3, b_2, b_1, b_0$  and  $C_0$ .

Now, we'll create a separate circuit for carry calculation

Let's write T.T. first

| a | b | Cin | cout |
|---|---|-----|------|
| 0 | 0 | 0   | 0    |
| 0 | 0 | 1   | 0    |
| 0 | 1 | 0   | 0    |
| 0 | 1 | 1   | 1    |
| 1 | 0 | 0   | 0    |
| 1 | 0 | 1   | 1    |
| 1 | 1 | 0   | 1    |
| 1 | 1 | 1   | 1    |

PAGE NO. : \_\_\_\_\_

DATE : / /

→ When  $a=1$  &  $b=1$ , then no matter what's the  $c_{in}$ ,  $c_{out}$  will surely be 1. So  $a=1, b=1$  are called **carry generator (G)**.  $G = ab$ .

→ When  $a=1, b=0$  or  $a=0, b=1$ ,  $c_{in}$  gets propagated i.e.  $c_{out} = c_{in}$ . This is called **carry propagator (P)**.  
 $P = a\bar{b} + b\bar{a} = a \oplus b$ .

Not used

→ When  $a=0, b=0$ , then no matter what  $c_{in}$  is  $c_{out}$  will be 0, this is called **carry killer (K)**.  
 $K = \bar{a}\bar{b}$ .

So, we get  $c_{out}$  when either carry propagates or carry is generated. And to know  $G=ab$  &  $P=a \oplus b$ , we don't need any other i/p.

$$G_0 = a_0 b_0, P_0 = a_0 \oplus b_0$$

$$G_1 = a_1 b_1, P_1 = a_1 \oplus b_1 \text{ and so on.}$$

Ans: The only basic gates can be used with delay of 1, then delay of  $P_3$ ?

⇒ 3s. (Only OR, NOT, AND delay)

→ Initially at  $t=0$   $a_i, b_i, c_0$  are available

then we can get  $G_i$  at  $t=1$  &  $P_i$  at 2s.

So at 2 we have all  $G_i$ 's & at 3s we have all  $P_i$ 's

if, 2 is delay of basic gates and of XOR is 3s



PAGE NO.:

DATE: / /

→ Now we have  $a_i, b_i, P_i$  and  $G_i$ . Now we need to generate  $c_{i+1}$  from them. i.e. Cout from stage  $i$ .

$$C_{i+1} = (a_i b_i) + C_i (a_i \oplus b_i)$$

↘ Formulae we already know from FA.

$$C_{i+1} = G_i + C_i P_i$$

Intuitively also for  $C_{i+1}$  either gen. at  $i$  or prop.  $C_i$ .

$$C_1 = G_0 + P_0 C_0 \rightarrow \text{calculated from initial data}$$

$$C_2 = G_1 + P_1 C_1 \rightarrow G_1, P_1 \text{ available.}$$

$$C_2 = G_1 + P_1 (G_0 + P_0 C_0) \rightarrow \text{Cal. from initial data.}$$

⋮

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

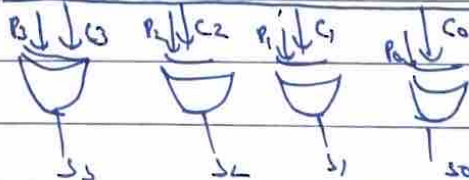
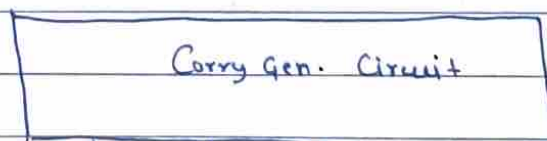
↘ From initial data.

→ So we can gen. all carries initially so that we don't have any dependency from prev. stage.

$$\text{Now } S_0 = a_0 \oplus b_0 \oplus C_0 = P_0 \oplus C_0$$

$$S_i = a_i \oplus b_i \oplus C_i = P_i \oplus C_i$$

So, CLA may look as:



OR, So is it and everyone propagates, C.C.P.B.P.

$$\text{Ja Cu} = C_0 + C_1 P_1 + C_2 P_2 + C_3 P_1 P_2 + C_4 P_1 P_3 + C_5 P_2 P_3 + C_6 P_1 P_2 P_3$$

Way any  
term is  $4i$

•  $C_i$  has it+1 literals at most in 1 term So far-in reg  
for AND gate of  $C_i$  is it+1

S.  $\text{for-in } q, \text{ OR} \rightarrow \{t\}, \text{ for-in } q, \text{ AND} \rightarrow \{t\}$   
 $\# \text{ OR} \rightarrow 1, \# \text{ AND} \rightarrow 1$

⇒ Delay of CLA:

Q. → When fan-in is unlimited and delay of basic gates is  $d$ .

Here we get  $P_i$  at 3rd &  $C_i$  at 5

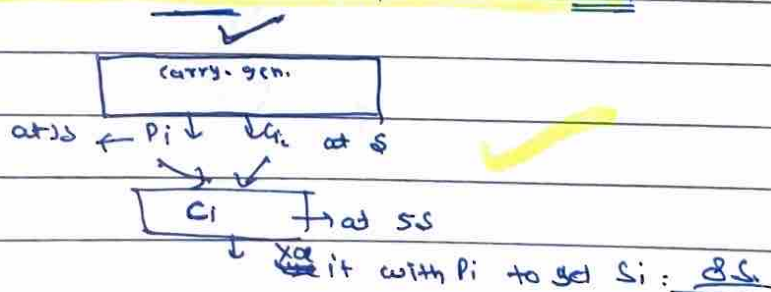
Now we get  $C_{i+1}$  after 2s as 1 level of AND.

1 level of OR,  $\infty$  unlimited

So, at 5th we have  $P_i, C_i \in C_i$

Now we need to XOR  $C_i$  &  $P_i$ , which will take 32

So at 8d we have our sum





PAGE NO. :

DATE : / /

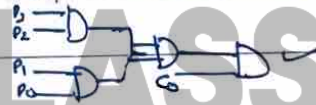
**NOTE**  $\leftrightarrow$   $C_0$  is not always zero, bcz, we only manufacture adder of fix bits (say 8 bit adder), then we add 16 bits by using it twice, so, while using it 2nd time  $C_{in}$  may not be zero.

$\Rightarrow$  Delay of  $n$  bit CLA when fan-in is 2 and delay of basic gates is  $\Delta$ .

$\rightarrow$  we'll get  $Q_i$  at 5 and  $P_i$  at 3. (like before), as just need ~~5~~ to do XOR, AND.

$\rightarrow$  Now we'll get  $C_4$  after.

Crit. critical path will be  $B, P_2, P_0, C_0$ , so  
3 levels of AND



and then 5 OR. gate i/p.  $\rightarrow$  3 levels for this

So get  $C_4$  after 6  $\Delta$ s.

so finally  $S_4$  will be available at  $\frac{3\Delta + 6\Delta + 3\Delta}{3} = 4\Delta$

Approx  $\Theta(\log n)$  levels for AND/OR gates if fan-in can't be unlimited. Base of  $\log$  is fan-in.

$\rightarrow$  GRE que. In Google doc.

PAGE NO. : \_\_\_\_\_

DATE : / /

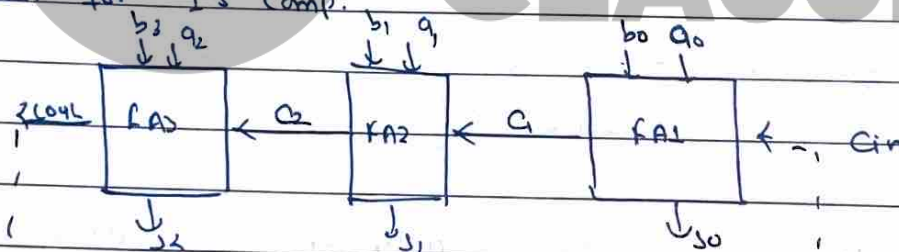
So far, we have seen adders for unsigned addition  
i.e.  $O(n)$  for RCA &  $O(\log n)$  for CLA if fan-in is limited.

Now, for signed addition.

a) 1's Comp: I/P in 1's comp (A, B)  
Sum  $A+B$  using adder, adder(A, B)  
We get  $S$  &  $cout$ , sum them, adder(S,  $cout$ )  
This is  $A+B$  in 1's comp form.  
 $2 \times O(n)$

b) 2's Comp: I/P in 2's comp (A, B)  
Sum  $A+B$  using adder add(A, B)  
Ignore  $cout$ , this is final result in 2's comp. form  
 $O(n)$  ✓

→ Circuit for 1's comp:



↳ Wrapping end carry.

↳ Not a seq. circuit, just using 1  
circuit twice.



*These notes are the property of GOClasses. Please do not share them without their consent.*

*For any questions or issues regarding my handwritten notes, feel free to reach out via email at [karan757527@gmail.com](mailto:karan757527@gmail.com) or DM me on telegram at @karan757527 (or by clicking [here](#)).*

*I would love to know if you liked my notes. Your feedback and appreciation are invaluable, so don't hesitate to share your thoughts. Click [here](#) to connect with me on LinkedIn or you can find me as @agrawalkaran.*

*Wishing you success on your journey ahead!*

*- Karan Agrawal*